

PROJEKTNA SPECIFIKACIJA

SISTEMA: WANDERLUST

1. Uvod

Wanderlust je napredna mikroservisna platforma dizajnirana da poveže turističke vodiče i turiste. Sistem omogućava vodičima kreiranje kompleksnih tura sa geografskim ključnim tačkama, dok turistima pruža mogućnost istraživanja, kupovine i interaktivnog izvršavanja tura uz pomoć simulatora lokacije. Pored osnovnih funkcionalnosti, platforma integriše društvene elemente kroz blogove i sistem praćenja korisnika.

2. Arhitektura Sistema

Sistem je zasnovan na **mikroservisnoj arhitekturi** sa ciljem postizanja visoke dostupnosti i skalabilnosti.

2.1. Tehnološki okvir

- **API Gateway:** Izolovana ulazna tačka koja vrši ruting zahteva ka odgovarajućim servisima.
- **Komunikacija:**
 - Eksterna: REST protokol.
 - Interna: gRPC protokol za kritične performanse i SAGA obrasci za distribuirane transakcije.
- **Baze podataka:**
 - **Relacione (PostgreSQL):** Za transakcione podatke (korisnici, ture, narudžbine).
 - **Document-based NoSQL (MongoDB):** Za fleksibilne strukture (npr. sadržaj blogova).
 - **Graph NoSQL (Neo4j):** Za upravljanje vezama između korisnika i sistem preporuka.
- **Kontejnerizacija:** Svaki servis je izolovan u sopstvenom Docker kontejneru sa nezavisnom bazom podataka.

3. Uloge u sistemu

1. **Administrator:** Upravlja korisničkim nalogima, vrši monitoring sistema i blokiranje neadekvatnog sadržaja/korisnika.
2. **Vodič:** Kreator tura i edukativnog sadržaja.
3. **Turista:** Kupac tura, učesnik u turama i autor blogova.

4. Funkcionalni Moduli

4.1. Upravljanje korisnicima i profilima (Stakeholders Service)

- **Registracija i Autentifikacija:** Podrška za uloge Vodiča i Turiste. Administratori se definišu na nivou baze.
- **Administracija:** Pregled svih registrovanih naloga i mogućnost suspenzije (blokiranja) korisnika.
- **Personalizacija:** Svaki korisnik poseduje profil koji uključuje biografiju, citat, profilnu sliku i osnovne informacije. Podržana je izmena podataka od strane vlasnika profila.

4.2. Blog i Društvena interakcija (Blog Service)

- **Kreiranje sadržaja:** Korisnici mogu kreirati blogove koristeći **Markdown** jezik za formatiranje opisa. Blog sadrži naslov, datum i opcione vizuelne priloge.
- **Interakcija:** Sistem omogućava ostavljanje komentara i lajkovanje objava.
- **Ograničenja:** Mogućnost komentarisanja bloga može biti uslovljena statusom pratioca (definisano kroz Follower servis).

4.3. Sistem pratioca (Follower Microservice)

- **Graf povezanost:** Implementiran pomoću **Neo4j** baze podataka.
- **Funkcionalnosti:**
 - Zapačivanje drugih korisnika.
 - **Feed filtriranje:** Korisnici mogu videti i komentarisati blogove samo onih korisnika koje prate.
 - **Sistem preporuka:** Algoritam preporučuje nove profile za praćenje na osnovu zajedničkih veza (npr. "ljudi koje prati osoba koju vi pratite").

4.4. Upravljanje turama (Tour Service)

- **Životni ciklus ture:** Tura prolazi kroz stanja: *Draft* (u pripremi), *Published* (objavljena) i *Archived* (arhivirana).
- **Kreiranje ture:** Unos naziva, opisa, težine, tagova i određivanje cene.
- **Ključne tačke (Keypoints):** Definisanje tačaka na mapi (latituda/longituda) sa nazivom, slikom i opisom. Sistem automatski kalkuliše ukupnu dužinu ture u kilometrima.
- **Tehnički uslovi za objavu:** Tura mora imati bar dve ključne tačke i definisano vreme obilaska za različite tipove transporta (peške, bicikl, automobil).

4.5. Prodaja i Shopping Cart (Purchase Service)

- **Korpa:** Turista dodaje ture u korpu koja dinamički računa ukupnu cenu.
- **Checkout proces:** Implementacija **SAGA obrasca** za osiguravanje konzistentnosti između naplate i dodeljivanja prava pristupa turi.
- **Tour Purchase Token:** Nakon kupovine, turista dobija trajni token koji mu omogućava pristup detaljnim informacijama o turi (svim ključnim tačkama).

4.6. Izvršavanje tura (Tour Execution Service)

- **Simulator pozicije:** Funkcionalnost koja simulira realno kretanje turista na mapi definisanjem trenutnih koordinata.
- **Praćenje sesije:** Kreiranje sesije (TourExecution) koja prati progres turista.
- **Provera blizine:** Sistem na svakih 10 sekundi proverava da li se klijent nalazi u radijusu ključne tačke. Ukoliko je uslov ispunjen, tačka se označava kao kompletirana sa zabeleženim vremenom.
- **Recenzije:** Nakon ili tokom ture, turista može ostaviti ocenu i komentar sa slikama.

5. Infrastruktura i DevOps

5.1. Dockerizacija

- Svaki mikroservis poseduje sopstveni Dockerfile.
- docker-compose orkestracija objedinjuje sve servise, baze podataka (PostgreSQL, MongoDB, Neo4j) i API Gateway u jedinstvenu mrežu.

5.2. Observability (Nadzor sistema)

- **Distributed Tracing:** Implementacija praćenja zahteva kroz sve mikroservise (npr. Jaeger) radi lakše dijagnostike uskih grla.
- **Centralizovani Logovi:** Agregacija logova svih servisa na jednom mestu radi brze pretrage grešaka.
- **Monitoring i Metrike:**
 - **Host metrike:** Praćenje resursa mašine (CPU, RAM, Mrežni saobraćaj).
 - **Container metrike:** Detaljan uvid u performanse svakog pojedinačnog kontejnera.

5.3. Distribuirane transakcije

- Primena **SAGA pattern-a** za operacije koje zahtevaju atomičnost nad više servisa, osiguravajući da sistem ostane u konzistentnom stanju čak i u slučaju otkazivanja jednog od mikroservisa tokom procesa (npr. tokom kupovine ili aktivacije ture).